

HW 12

Sam Ly

April 30, 2026

<https://github.com/samlyme/Assignments/tree/cs4080/ch24/main/CS4080>

Chapter 24

1. Reading and writing the `ip` field is one of the most frequent operations inside the bytecode loop. Implement the optimization of caching it in a local C variable. Write a couple of benchmarks and say whether the extra complexity is worth it.

The main change is to keep the current instruction pointer in a local variable inside `run()`:

```
static InterpretResult run() {
    CallFrame* frame = &vm.frames[vm.frameCount - 1];
    register uint8_t* ip = frame->ip;

#define READ_BYTE() (*ip++)
#define READ_SHORT() \
    (ip += 2, (uint16_t)((ip[-2] << 8) | ip[-1]))
```

Any opcode that changes control flow now changes the local `ip`, not `frame->ip`. For example, `OP_JUMP` adds to `ip` and `OP_LOOP` subtracts from it. The important bookkeeping is around calls, returns, and errors. Before calling another function, the VM stores the local value back into the current frame, then reloads `ip` from the new top frame after the call:

```
case OP_CALL: {
    int argCount = READ_BYTE();
    frame->ip = ip;
    if (!callValue(peek(argCount), argCount)) {
        return INTERPRET_RUNTIME_ERROR;
    }
    frame = &vm.frames[vm.frameCount - 1];
    ip = frame->ip;
    break;
}
```

The same reload is needed after `OP_RETURN`. Runtime error paths also need `frame->ip = ip;` immediately before calling `runtimeError()`, otherwise the line number reported for the error can be wrong.

I would benchmark one call-heavy program like recursive Fibonacci and one loop-heavy program that mostly executes arithmetic and jumps. This optimization should help both, but it especially helps the call-heavy case because `ip` is touched constantly while dispatching bytecode. A single digit speedup is enough to be worth it here because the added complexity is localized to the VM dispatch loop and does not affect the language semantics.

2. Native function calls are fast because they do not validate the number of arguments. Add arity checking. I would make native functions carry their expected arity the same way `ObjFunction` already does. Instead of storing only the function pointer in the native object, store both the C function and an integer arity:

```
typedef struct {
    Obj obj;
    NativeFn function;
    int arity;
} ObjNative;
```

Then `defineNative()` takes an arity argument, and `callValue()` checks it before invoking the native:

```
case OBJ_NATIVE: {
    ObjNative* native = AS_NATIVE(callee);
    if (argCount != native->arity) {
        runtimeError("Expected %d arguments but got %d.",
                     native->arity, argCount);
        return false;
    }

    Value result = native->function(argCount, vm.stackTop - argCount);
    vm.stackTop -= argCount + 1;
    push(result);
    return true;
}
```

This adds one integer comparison to every native call. That is a reasonable cost because it prevents native code from reading missing arguments out of uninitialized stack slots.

3. Extend the native function system so a native function can signal a runtime error. How does this affect native call performance?

The cleanest design is to separate the success signal from the returned Lox value. I would change `NativeFn` to return a boolean and have the native write either its result or an error message into the stack slot before the first argument:

```
typedef bool (*NativeFn)(int argCount, Value* args);

static bool clockNative(int argCount, Value* args) {
    args[-1] = NUMBER_VAL((double)clock() / CLOCKS_PER_SEC);
    return true;
}

static bool sqrtNative(int argCount, Value* args) {
    if (!IS_NUMBER(args[0])) {
        args[-1] = OBJ_VAL(copyString("sqrt() expects a number.", 24));
        return false;
    }

    args[-1] = NUMBER_VAL(sqrt(AS_NUMBER(args[0])));
    return true;
}
```

The VM then keeps the return value in the callee's old stack slot on success, or prints the message from that slot on failure:

```
if (native->function(argCount, vm.stackTop - argCount)) {
    vm.stackTop -= argCount;
    return true;
}

runtimeError(AS_CSTRING(vm.stackTop[-argCount - 1]));
return false;
```

This adds a branch to every native call, so a native-call microbenchmark would get slower. I still think it is the right tradeoff because a real standard library needs to reject bad dynamic values safely instead of letting C code crash or read invalid memory.

4. Add more useful native functions. What did you add, and how do they affect the feel of the language?

I would add a small math and utility base:

- `sqrt(n)` for square roots.
- `floor(n)` for rounding down.
- `pow(a, b)` for exponentiation.
- `str(value)` for converting simple values to strings.
- `type(value)` for debugging dynamic values.

Example Lox program:

```
fun distance(x1, y1, x2, y2) {
    var dx = x2 - x1;
    var dy = y2 - y1;
    return sqrt(pow(dx, 2) + pow(dy, 2));
}

print distance(0, 0, 3, 4);
print "answer is " + str(42);
print type(clock());
```

These do not make the language more expressive in theory, but they make it much more practical. Without them, even basic numerical programs require awkward hand-written helpers. The main design pressure is that each native function needs clear arity and runtime type errors so the boundary between dynamic Lox and static C stays safe.

Chapter 25

<https://github.com/samlyme/Assignments/tree/cs4080/ch25/main/CS4080>

1. Change `clox` so it only wraps functions in `ObjClosures` when the function actually needs upvalues. Compare the complexity and performance with always wrapping functions.

The compiler can choose between `OP_CLOSURE` and `OP_CONSTANT` after `endCompiler()` returns the completed `ObjFunction`:

```

ObjFunction* function = endCompiler();
uint8_t constant = makeConstant(OBJ_VAL(function));

if (function->upvalueCount > 0) {
    emitBytes(OP_CLOSURE, constant);
    for (int i = 0; i < function->upvalueCount; i++) {
        emitByte(compiler.upvalues[i].isLocal ? 1 : 0);
        emitByte(compiler.upvalues[i].index);
    }
} else {
    emitBytes(OP_CONSTANT, constant);
}

```

The VM then has to accept either `ObjFunction` or `ObjClosure` in a call frame. That means the frame can no longer store only a closure pointer. It either stores a generic `Obj*`, or it stores both the callable object and the underlying `ObjFunction`. The latter keeps bytecode access simple:

```

typedef struct {
    Obj* callable;
    ObjFunction* function;
    uint8_t* ip;
    Value* slots;
} CallFrame;

```

The performance tradeoff is mixed. Programs with many plain functions avoid allocating unnecessary closure objects, so they can become faster and allocate less. Programs that heavily use closures do not gain much, and the VM grows more complicated because call sites and stack traces have to handle two callable representations. I would weight ordinary non-closure function calls more heavily than a synthetic closure-allocation benchmark, but I still would want real program benchmarks before committing to the optimization. My default choice would be to keep always wrapping functions unless the allocation cost shows up clearly in profiling.

2. How should Lox behave when closures capture a loop variable? Change the implementation to create a new variable for each loop iteration.

I think each iteration should get a fresh variable. This matches what programmers usually mean when they create closures inside a loop: each closure should remember the value from the iteration that created it.

The compiler can implement this by introducing a hidden inner scope around the loop body when the `for` loop declares a variable:

- (a) Remember the original loop variable's name and stack slot.
- (b) Before compiling the body, begin a new scope and create a local with the same name initialized from the outer loop variable.
- (c) Compile the body so closures resolve to the inner per-iteration variable.
- (d) After the body, copy the inner variable back to the outer loop variable so assignments in the body still affect the loop condition and increment.
- (e) End the inner scope, closing the captured variable if a closure refers to it.

Conceptually, this makes code like this:

```

for (var i = 1; i <= 2; i = i + 1) {
  fun closure() {
    print i;
  }
  save(closure);
}

```

behave more like each loop body starts with a fresh shadowing variable:

```

{
  var i = outerI;
  fun closure() {
    print i;
  }
  save(closure);
  outerI = i;
}

```

With this behavior, two closures created by two different iterations print 1 and 2, not the same final value.

3. Using closures, write a Lox program that models two-dimensional vector objects. It should have a constructor, coordinate accessors, and an addition method.

A simple way to model objects with closures is to return a dispatcher function. The dispatcher's closed-over variables are the object's fields, and messages select which method to return.

```

fun vector(x, y) {
  fun object(message) {
    fun add(other) {
      return vector(x + other("x"), y + other("y"));
    }

    if (message == "x") return x;
    if (message == "y") return y;
    if (message == "add") return add;

    print "unknown message";
    return nil;
  }

  return object;
}

var a = vector(1, 2);
var b = vector(3, 4);
var c = a("add")(b);

print c("x");
print c("y");

```

The constructor `vector(x, y)` returns a closure that captures `x` and `y`. Calling the closure with `"x"` or `"y"` reads a coordinate. Calling it with `"add"` returns another closure that takes the other vector and constructs the sum.